

Práctica Unidad 1 - Semana 3 - Metodología de Sistemas II (2026)

TP – Clean Code | Estudiante: Varela Santiago Octavio - Email Institucional: santiago.varela@tupad.utn.edu.ar

(Comisión 15 - DNI 38011156)

Informe: Clean Code y Refactoring Seguro

Aclaración Técnica

El código fuente refactorizado no se incluye en este informe. El código completo puede consultarse directamente dentro del proyecto.

Contexto y Objetivos (General)

El Hospital Central incorporó un módulo de facturación médica funcional, pero con deficiencias en legibilidad, cohesión y acoplamiento. El objetivo de este informe es documentar la aplicación de principios de Clean Code, técnicas de refactoring seguro y la revisión técnica del código para mejorar su mantenibilidad.

Parte A: Clean Code

Interpretación del Dominio y Funcionalidad

El código original emplea variables de una sola letra carentes de significado semántico. La refactorización traduce las operaciones al lenguaje del dominio médico-administrativo, definiendo valores para estudios médicos, totales acumulados, bonificaciones del 50% para profesionales de la salud, coberturas del 10% por obra social y deducciones fijas basadas en el historial de turnos. La funcionalidad original y los resultados del sistema se mantienen intactos, modificando únicamente la estructura interna.

Problemas Identificados en el Código Base

- **Nombres:** Método principal sinónimo de "calcular". No indica el dominio específico de facturación médica.
- **Nombres:** Variables de un solo carácter. Falta de contexto sobre profesionales, obras sociales o turnos.
- **Cohesión:** Múltiples responsabilidades. Un único método suma estudios y aplica tres tipos de descuentos.
- **Legibilidad:** Dificultad de comprensión. Uso de nombres sin sentido del dominio médico.
- **Errores:** Ausencia de validaciones. No se validan listas vacías ni nulas.
- **Acoplamiento:** Dependencia de primitivos genéricos. Uso de listas de decimales en lugar de modelos de dominio.

Estructura del Proyecto Propuesta

Se establece una separación clara entre datos y comportamiento. Se definen entidades de dominio específicas para pacientes, estudios médicos y turnos, centralizando la lógica de negocio en una clase de servicio dedicada.

Parte B: Refactoring Seguro

Técnicas Aplicadas

- **Extract Method:** Se dividió el método principal largo en métodos individuales con nombres descriptivos. Esto resuelve la acumulación de responsabilidades y permite la reutilización del código.
- **Introduce Parameter Object:** Se agruparon múltiples parámetros de distintos tipos en un objeto contenedor único. Esto soluciona el crecimiento indefinido de la firma del método y facilita la escalabilidad de los datos.
- **Move Method:** La lógica de negocio se trasladó a una clase de servicio específica. Esto evita que la clase original crezca sin límite, aísla el comportamiento y permite eliminar código sin estado.

Parte C: Code Review

Revisión Técnica

Pregunta de Revisión	Evaluación	Justificación
¿Los nombres son descriptivos?	No	Uso de métodos y variables sin contexto del dominio médico.
¿Las funciones poseen una única responsabilidad?	No	El método procesa la suma base y tres deducciones diferentes.
¿Existe duplicación innecesaria?	Parcial	Las estructuras condicionales son simplificables y falta abstracción.
¿El código es fácilmente extensible?	No	Incorporar nuevas reglas requiere modificar el código

Pregunta de Revisión	Evaluación	Justificación
		central, rompiendo el principio OCP.
¿Hay manejo correcto de errores?	No	Ausencia total de validación de datos de entrada.
¿Existe acoplamiento innecesario?	Sí	Acoplamiento a tipos primitivos en lugar de objetos de dominio.
¿La cohesión es adecuada?	No	Agrupación de lógicas dispares en un mismo bloque de ejecución.
¿Se respetan principios SOLID?	No	Violación directa del Principio de Responsabilidad Única y el Principio Abierto/Cerrado.
¿El código resulta legible?	No	Uso críptico de números mágicos sin explicación lógica de negocio.

Parte D: Feedback Profesional

Tras la revisión técnica, se identificaron áreas críticas de mejora respecto a la mantenibilidad del código. El método analizado centraliza el cálculo base y tres reglas de descuentos distintas, transgrediendo el Principio de Responsabilidad Única (SRP). Asimismo, el uso de variables de un solo carácter omite el contexto del dominio médico y afecta la legibilidad. Se exige extraer cada regla en métodos privados descriptivos y consolidar la entrada de datos mediante un patrón Parameter Object. Estas refactorizaciones favorecerán la extensibilidad del módulo y asegurarán la viabilidad de futuras modificaciones.